

Attestation-Optimized Data Availability

A Native DA Layer Specialized for the Attestation Workload

Ligate Labs Research, Working Paper v0.2

Date: 2026-05-22

Contact: hello@ligate.io

Abstract

General-purpose data availability layers (Celestia, EigenDA, Avail, Walrus, 0G) are tuned for blob-shaped workloads: shares of 4 KB or larger, periodic high-throughput windows, clients that want raw data back. The attestation workload has a different shape. Records are 200 to 800 bytes. Throughput is sustained, not bursty. Records are signature-heavy: a typical attestation is 5 to 15 percent payload and 85 to 95 percent signature. Light clients want history-aware queries (“did attester X sign in epoch Y ”), not raw byte retrieval. Forcing one workload onto infrastructure tuned for the other is doable; whether it is optimal is a different question.

This paper specifies an attestation-optimized DA layer with three departures from the general-purpose baseline: **per-schema indexed commitments** for cheap attester-history queries, **signature aggregation within commitment trees** for bandwidth amortization, and a **fee market integrated with PoUA’s adaptive τ_{burn} rebase** (PoUA §4.4.2 + v0.8 §4.4.3) so that DA-side and consensus-side fee dynamics co-calibrate.

Three contributions. First, a workload-model characterization (§3): measurements of attestation size distribution, throughput profile, ordering requirements, retention requirements, and query patterns under Themisra / Iris / Mneme workloads, sized against Celestia’s blob-shape assumptions. Second, the protocol specification (§5-§9): the DA validator set, the per-schema commitment-tree data structure, the consensus and sampling protocol, the light-client query API, and the integrated fee market. Third, migration-decision criteria (§12): explicit thresholds (sustained throughput, light-client growth, fee-cost ratio) that would justify the engineering investment of a Ligate-native DA layer over staying on Celestia.

The paper is positioned as a **long-horizon migration target**, not a near-term replacement. Ligate Chain stays on Celestia through v1; this work exists so a future migration decision can be made deliberately, not reactively. §1.9 makes this stance explicit; §12 specifies the criteria under which the migration becomes justified.

1. Introduction

1.1 The Workload-Mismatch Thesis

General-purpose data availability layers were designed for the rollup-blob shape. A typical rollup posts a single multi-megabyte blob to the DA layer per block, recoverable as raw bytes by light clients running data-availability sampling. This workload favors large shares (4 KB or more), periodic uploads, and bandwidth-amortized erasure coding. Celestia’s 4 KB share size, namespace Merkle trees, and target throughput at the multi-MB/s level are all calibrated to this shape.

The attestation workload looks nothing like a rollup blob. A single attestation is 200 to 800 bytes (less than a fifth of a Celestia share). Attestations arrive continuously rather than in periodic bursts (one per Themisra session-end, one per Iris agent action, one per Mneme transfer). The bytes are signature-heavy: a typical attestation is 5 to 15 percent application payload and 85 to 95 percent threshold-signature plus framing. Light-client query patterns are history-aware: “did this attester sign anything in epoch Y ” matters more than “show me the raw bytes of attestation X .”

The thesis of this paper: **for an attestation-native chain, a DA layer specialized for the attestation workload would outperform a general-purpose DA layer on the three axes that matter (per-attestation cost, light-client query latency, fee-market sovereignty),**

but the engineering cost is non-trivial and the migration trade-off is not yet justified at Ligate’s current scale. The paper specifies what specialization would look like and identifies the criteria under which migration becomes worth it.

1.2 Why Now

Ligate is currently on Celestia and intends to stay through v1. This paper is not advocacy for migration. It exists for three reasons.

First, **understanding what we would build if we built our own is the right way to evaluate Celestia’s fit**. The Celestia-fit question is hard to answer in the abstract; concrete specification of an alternative provides the contrast that makes the comparison meaningful.

Second, **Celestia governance and fee changes are out of our control**. Celestia could raise blob fees, change share format, deprioritize attestation-shaped workloads, or face network-level disruption. Having a designed-fallback (even a fallback we never deploy) reduces strategic risk.

Third, **the workload-mismatch question gets sharper as devnet traffic accumulates**. Themisra’s attestation volume, Iris’s per-second throughput requirements, Mneme’s light-client query patterns will all be empirically measurable at devnet scale. A specification that anticipates the measurements lets us evaluate Celestia’s fit against concrete numbers rather than hand-waved estimates.

The right time to design a fallback is before you need one. This paper is the design.

1.3 The Workload-Mismatch Problem

Three quantifiable mismatches between the attestation workload and Celestia’s blob-shape baseline:

Mismatch 1: Share size. Attestations are 200-800 bytes; Celestia shares are 4 KB or more. Either we pad each attestation to a share (wasting ~80% of share bandwidth) or we batch attestations into shares (adding inclusion latency proportional to batch size). Neither option is bad, but both are paying a cost the design was not built to handle.

Mismatch 2: Throughput shape. Attestations arrive continuously, one or a few per block (or per sub-block timing window if the chain ships fast confirmation). Celestia is tuned for bursty rollup posts (one large blob per block, possibly skipped blocks). Sustained-low-rate-many-records is a different operating point than bursty-high-rate-few-records; overheads (mempool churn, gossip patterns, fee dynamics) compound differently.

Mismatch 3: Query shape. Attestation light clients want “did attester X sign anything in schema σ at epoch t .” Celestia answers “show me bytes in this namespace at this height.” Bridging requires an indexing layer that translates query-by-attester-history into query-by-bytes-in-namespace. The indexing layer is cheap to build but adds operational overhead (state to keep, queries to serve, freshness guarantees to maintain).

Each mismatch is doable. The question §12 asks: under what conditions does the cumulative cost of accommodating these mismatches exceed the cost of specialization?

1.4 The Central Question

What does an attestation-optimized DA layer look like, what does it cost to build relative to staying on Celestia, and under what conditions does the

cost justify itself?

This paper answers the first by specifying the mechanism in §5-§9. The second by quantifying the engineering cost in §11.3. The third by stating the migration-decision criteria in §12 explicitly.

1.5 Approach in Brief

The specialized DA layer makes three departures from the general-purpose baseline.

Per-schema indexed commitment trees (§6.1) replace the flat byte-array share model. Each schema gets its own Merkle (or Verkle) subtree, indexed by attester identity. A query “did attester X sign in schema σ at epoch t ” resolves in $O(\log n)$ proof checks against the schema’s subtree at the epoch’s root, rather than $O(\text{namespace-shares})$ blob-retrieval-and-parse work.

BLS signature aggregation within commitment trees (§6.3) amortizes the signature-heavy payload. Where a single attestation might be 200B payload + 600B signature, a batch of 100 attestations from the same attester set under the same schema can share a single 600B aggregated signature, dropping per-attestation overhead to $\sim 210\text{B}$. Across realistic schema-mix distributions, this is a 2-3x bandwidth improvement.

Fee market integrated with PoUA’s τ_{burn} (§9) ensures DA-side and consensus-side fee dynamics co-calibrate. A unified-fee chain that runs its own DA layer can route DA fees through the same chain-wide burn fraction that PoUA §4.4.2 maintains, so the cost-to-grind floor includes DA fees. On Celestia, DA fees are separate from Ligate’s protocol fees; the cost-to-grind floor calibrates against only one side.

1.6 Contributions

The paper makes four contributions.

A **workload-model characterization (§3)**: explicit measurements (or modeled projections, where devnet data is not yet available) of the attestation workload’s size distribution, throughput profile, ordering requirements, retention requirements, and query patterns. The section serves as the empirical foundation for the §4 specialization argument.

A **protocol specification (§5-§9)**: validator set, share-and-chunk format with erasure coding, per-schema commitment trees with signature aggregation, consensus and sampling protocol, light-client query API, fee market integration with PoUA τ_{burn} .

A **security analysis (§10)**: DA security under standard sampling assumptions, bandwidth bounds for honest validators, comparison of adversary cost-to-attack across Celestia / EigenDA / native-DA under a unified threat model.

Migration-decision criteria (§12): explicit thresholds (sustained attestation throughput, light-client population growth, observed DA cost as fraction of protocol fees) that would justify the engineering investment of switching from Celestia to a Ligate-native DA layer.

1.6.1 Status of Claims Empirical or heuristic, requiring devnet validation:

- §3 workload-model numbers (size distribution, throughput, query mix) are *projected* based on the product roadmap; devnet measurements will refine them.
- §11 comparison-table claims about competitors are drawn from each system’s published specifications; we cite the source and version.

Bounded under stated assumptions:

- §10.1 DA security holds under standard data-availability-sampling assumptions plus honest-majority of DA validator set (matching Celestia’s assumptions).
- §10.2 bandwidth bounds assume honest validators serve light-client requests at the configured rate; partial denial-of-service is bounded but not zero.
- §9.3 burn-share interaction theorem holds under PoUA’s chain-wide τ_{burn} being adaptive (per PoUA §4.4.2); a static τ_{burn} would weaken the calibration.

Proven (formal mathematical argument under standard cryptographic and BFT assumptions):

- §10 BFT safety under honest-majority validator set: standard Tendermint-style argument; documented for completeness.
- §6 commitment-tree structure: per-schema Merkle tree with signature-aggregation is well-formed under standard cryptographic hash assumptions.

1.7 Scope and Non-Goals

In scope:

- Attestation-shaped DA: small records, sustained throughput, signature-heavy, attester-history queries
- Per-schema indexing for cheap light-client queries
- Signature aggregation for bandwidth amortization
- Fee market integrated with PoUA τ_{burn}
- Migration-decision criteria

Explicitly out of scope:

- **Alternative consensus protocols.** This paper adopts CometBFT-style consensus by default (proven, audited, well-understood). Investigating HotStuff variants, Ethereum-style proposer-builder separation, or async-BFT is separate work.
- **Execution-layer DA.** Ligate is not a rollup framework. This DA layer serves Ligate’s own consensus-layer needs; rollups that want to post to it can do so but the design is not optimized for rollup-blob workloads.
- **Cross-DA bridging.** A future paper could specify mechanisms to bridge attestations from this DA layer to other DA layers (Celestia, EigenDA). Out of scope for v0.2.
- **Generic Tendermint-replacement.** The specialization is for attestation-shape; we are not redesigning consensus from scratch.

1.8 Document Structure

Section 1.6.1 separates proven, bounded, and empirical claims. Section 2 surveys five general-purpose DA layers (Celestia, EigenDA, Avail, Walrus, 0G) plus permanent storage networks. Section 3 characterizes the attestation workload empirically. Section 4 argues why specialization is justified (and where it is not). Sections 5-9 specify the protocol (validators, data structure, consensus, light-client, fee market). Section 10 analyzes security. Section 11 compares against Celestia and hybrid mode. Section 12 names the migration-decision criteria. Section 13 lists limitations; section 14 concludes.

1.9 Stance: Not Advocacy

This paper is not advocacy for migration off Celestia. Ligate Chain stays on Celestia through v1. The four flagship products at v1 (Themisra, Mneme, Iris, Atlas) operate on Celestia DA without disruption.

The paper exists because designing a specialized fallback before you need one is the right discipline. The §12 criteria explicitly state the thresholds (sustained throughput, light-client growth, fee-cost ratio) under which a future migration decision becomes worth considering; until those criteria are met, this paper is reference material, not a roadmap.

Three explicit non-claims:

1. **Celestia is not failing.** Celestia is production-grade and well-suited for many workloads. It is not optimally suited for an attestation-only chain, but “not optimally suited” is not the same as “broken.”
2. **Migration is not imminent.** Even if §12 criteria become satisfied, migration would be a multi-quarter engineering project. We would not start until the criteria are clearly met.
3. **This paper does not commit Ligate to building a DA layer.** It documents what we would build if we did. Future organizational decisions are separate from this paper.

2. Background and Related Work

This section surveys the six families of DA infrastructure that an attestation-native chain could plausibly use: five general-purpose DA layers (Celestia, EigenDA, Avail, Walrus, 0G) and two permanent-storage networks (Filecoin, Arweave). For each, we name what it does well and where the workload mismatch with attestations bites.

2.1 Celestia

Celestia (Al-Bassam et al., 2019; live since 2023) is a modular data availability layer using 2D Reed-Solomon erasure coding, namespace Merkle trees, and data-availability sampling. Block size targets 64 MB; share size is 4 KB or more. Validator set is decentralized (~150 validators at launch, growing). Light clients sample shares pseudo-randomly to confirm the data was published; under 50%-honest sampler assumptions, sampling 64 shares per block gives $\sim 2^{-32}$ false-acceptance probability.

What Celestia does well. Production-grade engineering. Well-audited cryptography. Decentralized validator set. Light-client UX that works (Celestia’s sampling clients are deployable). Reasonable throughput at multi-MB/s.

What Celestia does not do well for attestations. Share size (4 KB+ versus attestation-size 200-800B): forces padding or batching. Namespace queries answer “show me bytes in this namespace at height H” but not “did attester X sign in schema σ at epoch t”; the latter requires an off-DA indexing layer. Fee market is denominated in TIA and adjusts to Celestia’s chain-wide congestion, not Ligate’s per-schema dynamics; cost-to-grind floor calibrates only against Celestia, not Ligate.

Ligate Chain v0 currently uses Celestia as its DA layer (specifically, Mocha-testnet for `ligate-devnet-1`). The plan for v1 is to remain on Celestia; this paper specifies the fallback

target.

2.2 EigenDA

EigenDA (EigenLayer Labs, 2024) is a restaking-based DA layer on EigenLayer. Operator set is sized to a slashing budget (in restaked ETH); throughput target is ~10 MB/s per the public specifications. Adopted by several Ethereum L2 rollups.

What EigenDA does well. Ethereum-anchored economic security. Operators inherit slashing from EigenLayer’s restaking primitive. Integrates cleanly with Ethereum L2 ecosystem (deposit on Ethereum, post to EigenDA, settle on Ethereum).

What EigenDA does not do well for Ligate. ETH-denominated economic security does not align with \$AVOW-denominated PoUA reputation; the security models live in different economic stacks. Operator set is shared with other AVS workloads (operators serve EigenDA plus other restaking-based services); not Ligate-dedicated. Operator-set governance is EigenLayer’s, not Ligate’s. Fee market is ETH-denominated, exchange-rate risk to anyone billing in \$AVOW or USD.

2.3 Avail

Avail (Polkadot ecosystem spinout, live 2024) is a DA layer using KZG polynomial commitments and validity proofs. KZG commitments are succinct (constant-size regardless of data size), enabling cheap light-client verification.

What Avail does well. KZG succinctness: light clients verify with constant proof size. Validity-proof model: no fraud proofs needed at the DA layer. Polkadot-ecosystem integration.

What Avail does not do well for Ligate. Same share-size and query-shape mismatch as Celestia. Light-client benefit of KZG over Merkle is modest when the workload is small records (KZG proof verification cost exceeds the saved bandwidth at attestation sizes). DOT-ecosystem-coupled; bridging to Ligate requires its own infrastructure.

2.4 Walrus

Walrus (Sui Foundation, 2024) is a blob-storage layer in the Sui ecosystem with erasure coding and a Sui-anchored economic model. Designed for general object storage (NFT assets, file backups) more than real-time DA.

What Walrus does well. Object-storage UX. Sui-ecosystem integration.

What Walrus does not do well for Ligate. Not designed for real-time attestation throughput. Sui-ecosystem-coupled. The throughput-latency-cost profile is tuned for storing-objects-occasionally, not for stream-of-small-records. Brief mention; not a serious candidate for attestation DA.

2.5 0G

0G (live 2024) markets itself as an AI-focused DA layer with high-throughput claims (50 GB/s targets in marketing, though achieved throughput at production scale is the more conservative number). Claims attestation-friendliness via per-application namespace optimization.

What 0G does well. Workload-targeted marketing; closest peer in design philosophy to what this paper specifies. High-throughput claims if they hold up.

What 0G does not do well (or is unclear about). 0G’s technical specification is still evolving as of mid-2026; published details are limited. Validator-set decentralization, light-client protocol details, and fee-market specifics are not fully public. We track 0G as a peer and reference, but the comparison in §11 is based on what’s specified publicly, which may be incomplete.

2.6 Other Storage Networks (Filecoin, Arweave)

Filecoin and Arweave are permanent-storage networks, not real-time DA. Filecoin’s storage providers commit to long-term storage with proofs-of-replication; Arweave aims for permanent storage via a Proof of Access mechanism. Useful for **archival of historical attestations** (a Ligate node could pin old commitment-tree roots to Arweave for very-long-retention guarantees) but not for the hot DA path.

Mentioned here for completeness. The §6.5 tiered-retention design references these networks as the cold-storage tier; the hot DA tier is the work of this paper.

3. The Attestation Workload

This section specifies the workload model that motivates a specialized DA layer. The model has five components: per-schema size distribution (§3.1), throughput profile (§3.2), ordering requirements (§3.3), retention requirements (§3.4), and dominant query patterns (§3.5). Each component is formally specified with expected calibration targets; numerical values are recommended starting points subject to refinement against devnet measurements at v0.2.5+.

The workload model is the design baseline against which specialization decisions in §6 (data structure) and §10 (security analysis) are evaluated.

3.1 Size Distribution

The attestation size distribution per schema is the primary input to the share-size and erasure-coding decisions in §6.

Formal model. Let σ index registered schemas. Each schema’s attestation size S_σ is treated as a heavy-tailed positive random variable. The simulator and v0 calibration use a log-normal parameterization:

$$\log S_\sigma \sim \mathcal{N}(\mu_\sigma, \sigma_\sigma^2)$$

with per-schema parameters $(\mu_\sigma, \sigma_\sigma)$ derived from the schema’s payload structure. The schema registration declares an *expected median size* and a *p99 cap*; the runtime rejects attestations exceeding the cap, providing per-schema bandwidth predictability.

Component breakdown. A typical attestation carries:

- **Schema header** (~32 bytes): schema-id, version, attester-set-id reference
- **Payload digest** (~32-64 bytes): hash of off-chain content (the *what*)
- **Submitter address** (~32 bytes): who submitted
- **Attester signature** (~64-256 bytes): threshold signature from the attester set; size depends on signature scheme and threshold cardinality

- **Optional metadata** (variable): timestamps, references to other attestations (cross-schema composition #6), reveal-at fields (time-locked attestations #7)

The signature is the dominant size term. Ed25519 threshold signatures at threshold k and group size n are $\approx 64 + 32k$ bytes; BLS aggregations collapse this to 48 bytes regardless of k (at the cost of pairing-friendly curve operations during verification). Native DA can take advantage of BLS aggregation (§6.3) to reduce per-attestation signature payload.

Expected values per flagship schema.

Schema	Median (B)	p99 (B)	Driver of size
themisra.proof-of-prompt/v1	240	480	Threshold sig + prompt digest
mneme.tx/v1	300	600	Tx receipt: amount, recipient, sigs
iris.agent-action/v1	450	1100	Action + provenance refs
atlas.verification-result/v1	250	500	Verdict + artifact hash
atlas.crawl-record/v1	380	900	URL, content hash, crawler sig
chain.token-mint/v1	600	1500	Schema-bound-token mint event
atlas.user-report/v1	320	700	User-submitted content report

[Measured at v0.2.5+: these are the v0.1 expected values from architectural analysis. v0.2.5 will replace them with measured devnet medians and p99s after the first quarter of devnet attestation traffic. The schema registration mechanism will lock in per-schema p99 caps based on measured 99.5th percentiles plus a 50% margin.]

Comparison with Celestia. Celestia’s share-size minimum is 4 KB (`SHARE_SIZE` = 4096 in production parameters at the time of writing). All seven attestation schemas have median sizes below 700 B and p99 sizes below 1.5 KB. On Celestia, each share carries either one heavily-padded attestation (3-4× space waste) or multiple attestations packed without per-schema indexing (loses query-time schema-scoped sub-tree structure).

A native DA layer with 512-byte or 1-KB share sizes (§6.2) eliminates the padding problem at the cost of more shares per block, which is a controllable parameter rather than a fundamental cost.

3.2 Throughput Profile

The throughput profile drives the consensus block size, share count per block, and validator bandwidth requirements in §5 and §10.

Formal model. Per-schema attestation arrivals are modeled as independent Poisson processes:

$$N_{\sigma}(t) \sim \text{Poisson}(\lambda_{\sigma}t)$$

where λ_{σ} is the per-schema arrival rate. Aggregate chain throughput is the sum over schemas:

$$A(t) = \sum_{\sigma} \lambda_{\sigma}$$

Bursts vs sustained. A characterizing property of attestation traffic, distinct from blob-rollup traffic, is that $A(t)$ is **sustained** rather than bursty. Rollup chains post a single multi-MB blob per block at a frequency determined by block time; the inter-blob arrival is bursty by construction. Attestation arrivals are continuous: each user action (a ChatGPT prompt, a wallet transfer, an agent action) generates an attestation independently. The arrival rate has diurnal and weekly cycles but is otherwise steady.

This is closer to a high-write-rate database than a blob-storage workload.

Calibration targets.

Phase	Aggregate A (atts/sec)	Driver	Block size implication
v0 devnet	50-100	Themisra design partners + early adopters	~0.5 MB/block at 12-sec blocks (median size)
v1 mainnet (year 1)	200-500	Themisra + Mneme + Atlas launches	~2 MB/block
v2 mainnet (year 2-3)	1000-5000	Iris agent traffic dominates; 100s of agents per user	~20 MB/block at sustained peak
v3 mature	10000+	Full agent ecosystem; AI-action attestation per second per user	~100 MB/block; native DA becomes binding

[**Measured at v0.2.5+:** these are calibration targets, not predictions. Devnet measurement at v0.2.5 will firm up the v0 row; subsequent revisions will track aggregate A as adoption progresses.]

Variance and tail behavior. Under a Poisson process, the per-block attestation count N_{block} has standard deviation $\sqrt{\lambda \cdot E}$ where E is the block time. At $A = 1000$ atts/sec and $E = 12$ sec, expected block count is 12,000 with standard deviation ≈ 110 , a coefficient of variation $\approx 1\%$. This means block sizing can be tight (p99 block size $\approx 1.025 \cdot$ mean), a meaningful efficiency advantage over bursty workloads where p99/mean ratios of $5\times$ to $10\times$ are common.

Per-validator bandwidth. A validator must serve sample requests for the rolling DA window plus per-block tally work. At v3 sustained peak ($A = 10000$, 100 MB/block), per-validator bandwidth is dominated by block ingestion rather than DA sampling overhead. The native DA design (§7) provides the bandwidth-per-validator analysis under the stated workload.

3.3 Ordering Requirements

Ordering requirements interact with the consensus design (§7) and the slash-detection logic in PoUA’s §A.1 / §A.2.

Per-schema strict ordering. Within a single schema, the chain enforces a strict total order on attestations. This is required because:

1. **Slash detection** (PoUA §A.1 KL-divergence detector): the detector reads a per-validator distribution over schema-IDs included in proposed blocks. Without per-schema ordering, the distribution becomes ambiguous and the FPR bound in §A.4 weakens.
2. **Cross-attestation references** (cross-schema composition #6): a consumer schema referencing an input attestation needs the reference to resolve to a unique on-chain record. Strict per-schema ordering provides this.
3. **Time-locked reveals** (time-locked attestations #7): commit-reveal pairs are ordered by commit-height; without per-schema ordering, the reveal-window check is ill-defined.

Cross-schema eventual ordering. Across schemas, the chain provides only eventual ordering via block-height totality. Attestations of different schemas in the same block are partially ordered (any in-block tie-break suffices); attestations across blocks are ordered by block height.

Formal statement. Let $\rightarrow_\sigma$ denote the per-schema ordering relation and $\rightarrow_h$ the block-height ordering. The chain provides:

$$\forall \sigma : (a_1, a_2 \in \sigma, a_1 \rightarrow_\sigma a_2) \implies a_1 \neq a_2 \wedge (\text{height}(a_1), \text{seq}(a_1)) < (\text{height}(a_2), \text{seq}(a_2))$$

where $\text{seq}(a)$ is the per-schema in-block sequence number, and:

$$\forall a_1, a_2 : (a_1 \rightarrow_h a_2) \iff \text{height}(a_1) < \text{height}(a_2)$$

The relaxation from strict-total to per-schema-strict-plus-cross-schema-eventual matters because it permits the §6 commitment scheme to use per-schema sub-trees rather than a single global tree, reducing inclusion-proof size for schema-scoped queries.

Comparison with Celestia. Celestia provides strict total ordering via block height + namespace + share index. This is stronger than necessary for attestation; the savings are at the indexing layer (§6.1), not at the consensus layer.

3.4 Retention Requirements

Retention requirements drive the storage cost model and the tiered-pruning design.

Three-tier model. The native DA retention is tiered by access pattern.

Tier	Duration	Stored content	Verification surface	Cost driver
Hot	30 days minimum	Full attestation bytes + commitments	Full DA sampling	Bandwidth + storage
Warm	1 year	Per-schema commitment trees (no payload)	Inclusion proofs only	Storage

Tier	Duration	Stored content	Verification surface	Cost driver
Cold	Forever	Per-epoch summary commitments	Attestor-history queries only	Minimal storage

Hot tier supports recent-action verification: a user verifying their own ChatGPT prompt from yesterday, or a Mneme wallet showing recent transactions. Bandwidth-bound; needs full DA sampling.

Warm tier supports historical inclusion checks: a regulator auditing a 6-month-old attestation, or a creator economy claim against an older Themisra attestation. Storage-bound; payloads can be served from warm-tier validators or external archive nodes.

Cold tier supports attestor-history queries: “did X sign in schema σ at epoch t ?” answered by per-epoch summary commitments. Minimal storage cost (one commitment per schema per epoch); supports indefinite retention.

Storage cost analysis. Per-validator storage at v2 sustained peak ($A = 1000$, median size 400 B):

- Hot tier (30 days): $1000 \cdot 400 \cdot 86400 \cdot 30 \approx 1$ TB
- Warm tier (1 year): per-schema-commitment is ≈ 32 B per attestation; $1000 \cdot 32 \cdot 86400 \cdot 365 \approx 1$ TB
- Cold tier (forever): per-epoch summary is ≈ 32 B per schema per epoch; at 7 schemas, $86400/14400 \approx 6$ epochs/day $\times 32$ B $\times 365$ days $\times 7$ schemas ≈ 0.5 MB/year. Negligible.

Total per-validator storage at v2: ≈ 2 TB hot+warm + cumulative cold tier. Comparable to a single SSD; tractable.

Comparison with Celestia. Celestia provides full retention of all submitted blobs (subject to its own pruning policy at chain level). For attestation, this is over-retention: the warm and cold tiers do not need full payload retention, only commitment retention. Native DA’s tiered model captures this savings.

Pruning interaction with PoUA reputation. Reputation scoring depends on per-validator attestation history. The cold tier (per-epoch summaries) preserves what’s needed for reputation reconstruction without full payload retention. Validators not in the active set can re-derive their reputation from cold-tier summaries plus the chain history at any future point.

3.5 Query Patterns

The query patterns drive the light-client protocol design (§8).

Three dominant queries account for the bulk of light-client load:

Q1: Inclusion proof. “Prove that attestation a is included in the canonical chain.”

- Frequency: high (every attestation read triggers one)
- Verification cost: should be $O(\log N)$ in chain size or $O(\log d)$ in per-schema commitment depth

- Comparison: Celestia provides namespace Merkle proofs; native DA can provide tighter per-schema commitment proofs at $O(\log d)$ where d is schema-tree depth (typically much smaller than full chain depth)

Q2: Attestor-history query. “Did attester X sign in schema σ between epochs t_1 and t_2 ?”

- Frequency: medium (reputation verification, audit trails, slashing-related queries)
- Verification cost: should be $O(t_2 - t_1)$ summary commitments plus per-epoch attester-index lookups
- Comparison: Celestia does not natively index by attester; native DA’s per-schema commitment trees with attester-id sub-indexing answer in a few hundred bytes per query

Q3: Range query. “All attestations of type σ between heights H_1 and H_2 .”

- Frequency: low-to-medium (analytics, monitoring, compliance reporting)
- Verification cost: $O(H_2 - H_1)$ inclusion proofs plus the per-schema sub-tree commitments
- Comparison: Celestia answers via full namespace scan, $O((H_2 - H_1) \cdot \text{shares-per-block})$ in proof size; native DA reduces this by the per-schema indexing factor

Q1 dominates volume; Q2 dominates query-cost-savings benefit. Q1 is the high-volume, low-individual-cost query. Q2 is the query that motivates per-schema indexing: at any reasonable attestation rate, Q2 on a non-indexed DA layer becomes computationally expensive enough to push the lookup off-chain (creating a centralization vector). Native DA’s per-schema indexing keeps Q2 verifiable on-chain.

Privacy considerations. Attestor-history queries reveal which validator signed which attestations. This is necessary for reputation scoring and slash detection but creates a per-validator visibility surface. PoUA accepts this trade-off (the §A detectors require this visibility); a privacy-preserving variant is future work.

4. Why Specialization

Three quantitative arguments for specialization, plus one honest counter-list documenting where the case is weak.

4.1 Quantitative Mismatch with Celestia

A worked example. Themisra at peak load: 5,000 attestations per 12-second block at ~280B median size is ~1.4 MB per block of attestation payload. On Celestia, this becomes one of three options:

- **Option A: Pad to 4KB shares.** 5,000 attestations $\times$ 4096B = 20 MB per block, with ~93% padding. At Celestia’s per-byte fee, this is paying 15x what the actual payload requires.
- **Option B: Pack attestations into shares.** ~14 attestations per 4KB share, total ~360 shares per block. No padding waste, but loses per-attestation indexing: a “did attester X sign at height H ” query requires fetching and parsing all shares in the namespace, ~360 shares per block $\times$ 30-day retention = millions of shares to scan.
- **Option C: Hybrid with off-chain index.** Pack into shares (saves padding) plus an off-chain attester-history index (recovers query latency). Operational complexity, freshness guarantees, separate trust assumption.

Native DA avoids all three. Per-attestation share is ~280B native; per-schema indexed commitment tree (§6.1) answers attester-history queries in $O(\log n)$ proof verifications against a single epoch root. No padding waste; no off-chain index; no separate trust assumption.

Bandwidth savings. At peak load: ~1.4 MB native vs ~20 MB Celestia-padded (Option A) or ~1.4 MB raw plus the off-chain index cost (Option C). Native is dominant at scale.

4.2 Light-Client UX Cost

A “did attester X sign anything in schema σ during epoch t ” query has different costs across the design space.

On Celestia. Fetch the namespace Merkle proof for every block in the epoch (say, 300 blocks per 1-hour epoch at 12-second block time). For each block’s namespace proof, scan the included shares to check whether any attestation in schema σ came from attester X . Total work: 300 namespace fetches + share parses + signature checks. Practical bandwidth: tens to hundreds of KB per query at typical attestation density.

On native DA with per-schema indexing. Single inclusion proof against the schema’s commitment tree at the epoch root. The commitment tree is indexed by attester identity, so the proof is the attester’s leaf path. Practical bandwidth: ~1KB per query (one Merkle path).

The native UX is 1-2 orders of magnitude cheaper for the queries light clients actually want to run. For a wallet’s recent-activity feed, the difference is the difference between “snappy” and “noticeable lag.”

4.3 Fee-Market Sovereignty

Celestia’s per-byte fees are governance-controlled. In production they have moved twice in the last 18 months: once up (during the 2024 blob-fee debate), once down (post the 2025 throughput upgrade). Ligate’s per-attestation cost has thus moved twice without Ligate making any decision.

Native DA controls the full fee curve. The \$9 fee market couples DA pricing to PoUA’s τ_{burn} adaptive rebase, so DA-side fees and protocol-side fees co-calibrate. The chain’s cost-to-grind floor includes DA fees; the cost-to-grind floor on Celestia includes only Ligate’s protocol fees.

Sovereignty has a cost. Running our own DA layer means we set the fee schedule, but we also bear the validator-set operational risk and the audit burden. §11.3 quantifies this engineering cost; §12 specifies the criteria under which sovereignty is worth the cost.

4.4 Counter-Arguments (Why Not Specialize)

Honest list of arguments against specialization, weighted as they actually deserve:

(1) Engineering cost is multi-quarter. Designing, building, auditing, and bootstrapping a DA layer is real work. §11.3 quantifies: roughly 3-5 person-quarters of design + implementation + test + audit. This is substantial; the migration-decision criteria (§12) must clear this hurdle.

(2) Celestia is well-audited; native DA is not. Production-grade security on Celestia comes from years of operator experience plus multiple external audits. A from-scratch native DA inherits Sovereign SDK’s audit history but introduces new attack surface in the per-schema commitment tree and signature-aggregation paths. Audit cost is real and ongoing.

(3) Validator set bootstrapping is hard. A new DA validator set needs to be staked, slashed-against, geographically distributed, and resilient against operator failures. Celestia’s validator set is already there; using it is essentially free. Native DA needs to launch and grow a validator set, which is a coordination problem.

(4) Bridging cost during migration is non-trivial. Transitioning from Celestia to native DA during operation means attestations split across two DA layers for some window. Cross-DA bridging is feasible (§13.2) but adds protocol surface and operational complexity. The transition is a project on its own.

Net of arguments. Specialization is technically sound and has clear bandwidth + UX + sovereignty advantages. Whether those advantages exceed the engineering cost is a decision contingent on observed scale + Celestia evolution + Ligate’s strategic posture. §12 names the criteria; until they fire, “stay on Celestia” is the default.

5. System Model

5.1 Validators and the DA Validator Set

The native DA layer’s validator set is **shared with PoUA’s consensus validator set by default**. A validator that is in the PoUA validator set is also a DA validator and serves DA sampling requests. This shares the validator-set bootstrapping cost (one set, not two) and aligns economic security: an adversary attacking DA also faces PoUA reputation slashing.

Alternative: separate sets. A separate DA validator set (smaller, specialized to high-bandwidth operations) could be deployed in v1.5+ if (a) the consensus validator set becomes large enough that DA bandwidth budget is binding, or (b) governance preference favors specialized roles. v0.2 specifies the shared-set design and notes the separate-set extension; the migration between the two is a governance question, not a chain-design question.

Set size and threshold. Inheriting PoUA: $f < n/3$ Byzantine threshold, $n = 150$ at v1 target, $f = 49$. Honest-majority assumption transfers.

5.2 Shares, Chunks, and Erasure Coding

We adopt **2D Reed-Solomon erasure coding** as the sampling primitive, the same family Celestia uses. Reasons: well-understood, proven security under the standard sampling-protocol literature, light-client clients are deployable today.

Share size. We recommend **512 bytes or 1 KB** (vs Celestia’s 4 KB). At the median attestation size of ~280B, 1 KB shares fit 3 attestations per share with ~25% padding overhead; 512B shares fit 1-2 attestations with 10-40% overhead. Both are dramatic improvements over 4 KB Celestia shares’ 93% padding overhead at attestation sizes.

Block layout. A block contains $K \times K$ shares arranged in a 2D matrix, plus the $(K \times 2K) - K^2$ erasure-code parity shares produced by 1D Reed-Solomon extension along each row and column. Sample requests verify against any subset of the full $4K^2$ shares. With $K = 64$ (Celestia-compatible) and 1 KB shares, block size is $4 \cdot 64^2 \cdot 1024 = 16MB$ per block in extended form. Per-block attestation payload at ~1.4 MB at peak Themisra load is well within this budget.

Chunks. A chunk is the unit of network transport: a set of contiguous shares within a row. We use chunks of 16 shares (16 KB at 1 KB shares); standard gossip-network unit.

5.3 Light Clients

Three roles of light client:

Simple light client (single-claim verifier). Verifies a specific attestation’s inclusion against a published block header. Bandwidth: one Merkle path (~1 KB). Suitable for wallets verifying their own activity.

Historian (attestor-history verifier). Queries “did attestor X sign in schema σ during epoch t .” Bandwidth: one per-schema commitment-tree path (~1 KB) plus the per-epoch summary commitment (32 B). Suitable for activity-feed displays.

Full DA-sampling client. Samples shares pseudo-randomly across the full block to verify data availability. Bandwidth: k sample shares per block (typically $k = 8$ for 2^{-32} false-acceptance), ~8 KB per block sampled. Suitable for security-critical applications (regulatory monitors, audit tools).

Each role has its own bandwidth budget and trust assumptions. The simple light client trusts the validator set’s block headers; the historian additionally trusts the per-schema commitment-tree construction; the full sampler verifies availability cryptographically.

5.4 Adversary Model

Byzantine validators bounded by $f < n/3$. Standard CometBFT assumption. With $n = 150$, $f \leq 49$.

Bandwidth-bounded data-withholding. An adversary controlling some validator share might publish block headers without actually serving the data on request. DA sampling detects this: a client sampling shares pseudo-randomly will, with high probability, hit a withheld share and reject the block.

Eclipse attacks against light clients sampling. An adversary controlling a light client’s peer set can answer all sample requests with data the adversary knows is invalid. Defense: clients sample from independent peer sets, refresh peer sets periodically, and refuse to accept block headers without satisfying a sampling threshold.

Sample-amplification attacks. An adversary may try to make sample requests cheap to fabricate but expensive to verify (e.g., by serving shares that decode to garbage). Defense: the chain’s commit-then-reveal pattern (each share is committed to in the block header before clients sample) makes fabrication detectable.

6. Data Structure

6.1 Per-Schema Commitment Trees

Each schema gets its own commitment tree per epoch (where an epoch is configurable; default 5 minutes = 25 blocks at 12-second block time). The commitment tree is a Merkle tree indexed by attestor identity, with leaves containing the attestations submitted by that attestor in the epoch.

Commitment scheme choice. We default to **namespace-aware Merkle trees** (Celestia-style) for v0.2 recommendation. Reasons: (a) no trusted setup required (vs KZG which needs a trusted ceremony); (b) standard cryptographic assumptions (hash collision-resistance); (c) proof sizes are $O(\log n)$ but small in absolute terms for attestation-sized trees ($n \leq 10^4$ attestors per epoch, proof size ~ 400 bytes). KZG is the alternative if proof size becomes binding; Verkle is the longer-horizon path post-trusted-setup-deprecation.

Tree structure per epoch per schema:

- Leaves indexed by attessor identity (32-byte address).
- Inner nodes are SHA-256 of children.
- Per-attestor leaf is a sub-Merkle-tree of that attestor’s attestations in the epoch (so per-attestor history queries are efficient).
- Epoch root commits to all per-attestor sub-trees.

State storage. Per-epoch per-schema root is recorded in the block at the epoch boundary. Historians retain epoch-summary commitments forever; full-DA-sampling clients retain hot-tier shares for 30 days per §6.5.

6.2 Block Layout

Each block contains:

1. **Per-schema attestation shares** for attestations submitted in this block (variable shares, depending on per-schema activity in the block).
2. **2D Reed-Solomon erasure-coded extension** over the schema-share matrix.
3. **Per-schema partial commitment-tree updates** (the updated sub-tree paths for attestors who submitted in this block).
4. **Aggregate threshold signatures per schema per block** (BLS aggregation per §6.3).
5. **Cross-block summary references** for tiered retention (§6.5).
6. **Standard block header** (height, parent hash, validator-set signature, etc.).

Block size at peak: ~ 16 MB extended form ($4 \times K^2$ shares of 1 KB at $K = 64$), per §5.2.

6.3 Signature Aggregation

Per schema per epoch, all attestor-set signatures across attestations are aggregated into a single BLS signature of ~ 96 bytes (BLS12-381). This collapses the signature payload from $O(n_{\text{attestations}} \cdot 64)$ bytes to $O(1)$ per schema per epoch.

Verification per-attestation. Light-client inclusion proofs include the aggregation context: a Merkle path to the attestation plus the schema-epoch aggregate signature plus the per-attestor public-key list for that epoch. Verification cost is one pairing check ($\sim 1ms$ on modern CPUs).

Bandwidth saving. At peak Themisra (5,000 attestations per block, 64 B signature per): without aggregation, signature payload is $5000 \cdot 64 = 320$ KB per block. With aggregation, signature payload is ~ 1 KB per block (one BLS aggregate per schema-epoch, assuming ~ 10 active schemas at peak). 99.7% bandwidth reduction.

Proof-size cost. Aggregated signatures cannot be verified independently per attestation; the inclusion proof must carry the aggregation context. Typical inclusion proof size grows from ~ 500

bytes (Merkle path only) to ~1 KB (with aggregation context). Net of signature-payload savings vs proof-size increase, native DA is 100s of KB ahead per block.

6.4 Compression

Schema-aware compression on attestation payloads. Schemas share common prefixes: schema-id (constant per schema, ~16 B), attester identifier (32 B but often within a small set), timestamp/epoch (8 B with high temporal locality). Dictionary encoding compresses these prefixes to a few bytes per attestation.

Estimated saving: **20-40%** on raw payload bytes at typical attestation density. Compute cost: trivial (dictionary encoding is fast, no zlib-style overhead). Decompression at the light client: free in practice.

6.5 Tiered Retention

Three tiers of retention:

Hot tier (full DA sampling, 30 days). All blocks within the last 30 days are fully sampleable: all $K \times K$ shares plus extended parity. Validators serve sample requests at the §5.4 bandwidth target. Light clients verify DA before trusting block headers.

Warm tier (commitments only, 1 year). Blocks older than 30 days retain the per-epoch commitment-tree roots, signature aggregates, and block headers, but the underlying shares are pruned. Historians can verify attester-history queries against committed roots without needing share-level DA. New light clients cannot verify DA for warm-tier blocks; they trust the rolling chain header.

Cold tier (epoch summaries, forever). Blocks older than 1 year retain only per-epoch summary commitments (one 32-byte root per epoch per schema). Bandwidth-cheap; verifiable as an audit trail; sufficient to prove “yes, X attested at epoch Y” but not to retrieve the payload.

Migration between tiers. Per-epoch pruning runs at the tier boundaries. The chain’s runtime triggers pruning automatically; validators delete their local copies. Long-tail retention (cold-tier commitments) is small enough to retain forever (10^4 epochs/year $\times$ 32 B $\times$ 10 schemas = 3 MB/year).

Archive option. A schema can configure its commitments to also be pinned to Filecoin or Arweave (§2.6) as a permanent off-DA archive. This is application-layer.

7. Consensus and DA Layer Design

7.1 Consensus Choice

CometBFT-style consensus is the default. Reasons: (a) production-proven in dozens of Cosmos chains; (b) Sovereign SDK supports it natively; (c) Ligate already runs CometBFT-style consensus for PoUA. Adopting the same family for DA simplifies validator-set sharing (§5.1).

Alternative: Narwhal-Bullshark style for higher throughput. Narwhal-Bullshark decouples mempool/DA from consensus, achieving multi-second-to-sub-second latency at high throughput.

The cost: more complex implementation, less production track record. v0.2 picks CometBFT by default but notes Narwhal-Bullshark as the upgrade path if throughput becomes binding (post-v2).

7.2 Sampling Guarantee

Standard k -of- n DA sampling with $n = 2K \times 2K = 4K^2$ extended-form shares. Under 2D RS, a sample of $k = 8$ shares gives $\sim 2^{-32}$ false-acceptance probability (assuming honest-majority validator set), matching Celestia’s target. Higher k (16, 32) reduces false-acceptance further at linear bandwidth cost.

Light-client sample-rate target. 8 samples per block per client. At 16 MB extended block size: 8 samples $\times$ 1 KB = 8 KB per block sampled. A continuously-running light client at 12-second block time consumes ~ 700 B/s bandwidth.

7.3 Fork Choice

Standard CometBFT fork choice (longest chain with valid signatures). No deviation needed for the attestation workload; the DA layer’s fork choice matches the consensus layer’s.

7.4 Slot Timing

Block time: **12 seconds**, matching Ligate’s chosen consensus block time. Reasons: (a) shares PoUA’s slot boundaries (one consensus block = one DA block); (b) 12 seconds is enough to gossip all shares to ~ 150 validators in a typical network; (c) faster blocks (1-2 seconds) reduce attestation latency but multiply per-block overhead and tighten gossip schedule.

Epoch boundary. 25 blocks per epoch (5 minutes). Per-schema commitment-tree roots are committed at epoch boundaries; per-block partial updates are committed in-block.

8. Light-Client Protocol

8.1 Inclusion Proofs

A “did attestation a in schema σ get included” query returns: the Merkle path from the attestation leaf to the schema’s commitment-tree root, plus the schema-tree-root’s inclusion in the block header, plus the validator-set signature on the block.

Proof size: ~ 500 bytes (Merkle path) + ~ 32 B (commitment root) + ~ 96 B (BLS aggregate signature) = ~ 700 B. Verification: ~ 20 hash computations + ~ 1 pairing check, ~ 1 ms on commodity hardware.

8.2 Attestor-History Queries

A “show all attestations from attestor X in schema σ during epoch t ” query returns: the per-schema commitment tree’s leaf for attestor X at epoch t , plus the leaf’s sub-Merkle-tree containing X ’s individual attestations in the epoch.

Proof size: ~ 500 B for the path to X ’s leaf, plus the sub-tree (variable but typically ~ 200 B $\times$ number-of-attestations-by- X -in-epoch). Verification: $O(\log n_{\text{attestors-in-epoch}})$ hash computations.

Cross-epoch query. “All attestations from X in σ between epoch t_1 and t_2 ” returns one proof per epoch in the range. At a typical query of 7-day window with 5-minute epochs: $7 \times 24 \times 12 = 2016$ epoch proofs. Bandwidth-cheap (~ 1.4 MB total) but the most expensive light-client query in the protocol.

8.3 Epoch Summaries

Per-epoch summary commitment is the hash of all schemas’ commitment-tree roots at the epoch boundary. ~ 32 bytes per epoch. Historians use these to sync the chain header forward without replaying every attestation: download the chain header (epoch summaries every 5 minutes), verify validator-set signatures, accept.

Lightweight historians. A historian following 1 schema at 5-minute epochs adds 32 B per epoch to its state, ~ 3 MB/year. Tractable on mobile.

8.4 Fraud / Validity Proofs

If a validator publishes a block header committing to an invalid block (e.g., a malformed erasure code, an invalid signature aggregate, a missing attester leaf), any honest client can construct a **fraud proof**: a witness showing the inconsistency, plus the offending block header’s signatures. The fraud proof is gossiped to other validators, who reject the bad block.

Standard fraud-proof construction. v0.2 specifies the canonical encoding (Merkle-path-plus-inconsistency-witness format).

Validity-proof variant. A future v1+ extension could replace fraud proofs with succinct validity proofs (SNARKs over the block’s well-formedness). This would let light clients verify block validity in constant time, regardless of block size. The cost: SNARK proving overhead on the validator side ($\sim O(\text{seconds})$ per block on commodity hardware in 2026). Out of scope for v0.2; named in §13.

9. Fee Market

9.1 Pricing Model

The fee market is **hybrid**: per-byte cost (DA storage / bandwidth) + per-attestation cost (protocol fee, identical to PoUA on Celestia today).

Per-byte cost recovers DA-layer expenses: bandwidth to gossip shares, sampling-request servicing, light-client query-cost. At typical attestation size (~ 280 B median, ~ 1500 B p99): per-attestation DA cost = ~ 280 B $\times$ per-byte rate. At a target DA rate of 1 $\mu\text{AVOW}/\text{byte}$: ~ 280 μAVOW per attestation in DA cost.

Per-attestation cost is the protocol fee. Identical to per-schema-fees paper §3.2: b_σ per schema, subject to PoUA’s τ_{burn} and the schema’s ρ_σ routing.

9.2 Schema-Priced or Uniform?

Per-byte DA cost is **uniform across schemas**. Reasons: (a) DA cost reflects bytes-on-wire, which doesn’t care which schema; (b) per-schema differentiation lives on the protocol side (per-schema-fees paper §4); (c) uniform pricing avoids cross-schema fee-market complexity.

Per-attestation protocol fees are **per-schema**, governed by the per-schema-fees mechanism. Clean separation: DA-layer fees track bytes; protocol fees track per-schema economics.

9.3 Burn-Share Interaction

PoUA’s τ_{burn} rebase (PoUA §4.4.2 + v0.8 §4.4.3) operates on **protocol fees**, not DA storage costs. The reason: DA storage costs are payment for service rendered (bandwidth, storage); burning them would mean validators serve DA but earn nothing for it.

Cost-to-grind floor under native DA. PoUA Lemma 1 bounds the adversary’s cost-to-grind by $\tau_{\text{burn}} \cdot \Delta r / (\eta \cdot \alpha_{\text{eff}})$. Native DA does not change τ_{burn} , Δr , η , or α_{eff} ; the floor is identical to the Celestia-based deployment. The DA storage cost is a separate cost-per-attestation, paid in addition to protocol fees, that doesn’t enter the cost-to-grind argument.

9.4 Validator Reward Split

Validator income decomposes into:

1. **Protocol-fee reward:** $(1 - \tau_{\text{burn}}) \cdot (1 - \rho_{\sigma}) \cdot b_{\sigma}$ per attestation included. Same as PoUA on Celestia + per-schema-fees v0.2 §3.2.
2. **DA-fee reward:** per-byte rate $\times$ bytes-served. Pays for bandwidth + storage.
3. **Block reward:** protocol-set constant per block. PoUA §6.1.

Total validator income per block depends on: block contents (attestation count + sizes), block proposer status (gets tips), and validator weight in PoUA’s reputation.

Native DA does not change protocol-fee accounting. The §6.1 income decomposition transfers. What’s new is the per-byte DA reward, which is a separate stream funded by DA users (the chain’s protocol fee buyers).

10. Security Analysis

10.1 DA Security Model

The data-availability guarantee is the standard one for 2D Reed-Solomon-based DA layers: under honest-majority of the DA validator set (per §5.4) and a sufficient sampling rate per light client (per §7.2), data is available with high probability. The argument transfers directly from Celestia / Avail; we adopt their security model and parameter choices.

Formal claim. Under $f < n/3$ Byzantine validators and DA sampling rate $k \geq 8$, the probability that an honest light client accepts an unavailable block is bounded by 2^{-32} . Proof: standard 2D RS sampling argument. Sample-rate can be tuned up ($k = 16, 32$) for security-critical applications.

10.2 Bandwidth Assumptions

Validators must serve sample requests at the chain’s target light-client population times the sample rate. At v1 target: 150 validators, 10,000 light clients, 8 samples per client per block, 1 KB per sample. Per-validator bandwidth: $10,000 \times 8 \times 1024 / 12 / 150 \approx 4.4$ KB/s ambient sampling load. At peak Themisra (5,000 attestations/block, 16 MB extended block size), each validator gossips ~ 1.3 MB/s of new data plus serves the ambient sampling traffic.

Sustained-throughput target: **10 MB/s per validator** during peak load. Comparable to Celestia validators; achievable on commodity hardware.

10.3 Adversary Model

Same as §5.4. Byzantine validators bounded by $f < n/3$, plus bandwidth-bounded data-withholding, plus eclipse attacks, plus sample-amplification. Defenses per §5.4.

10.4 Comparison Under Unified Threat Model

We compare native DA against the §2 comparators on three axes that matter for security.

System	Validator threshold	Sampling guarantee	Security model	Avail latency	Workload-fit
Celestia	2/3 honest, 150 validators	2D RS, k -sample	Honest-majority validators	12s	General-purpose (blob-shaped)
EigenDA	Restaked operators (variable)	Custom DA proofs	Restaking-budget bounded	3s	Ethereum L2 rollup blobs
Avail	2/3 honest validators	KZG validity proofs	Validity-proof model	Sub-second	General-purpose, KZG-succinct
Walrus	Fixed committee, Sui-anchored	Erasure-coded blobs	Honest-supermajority	Variable	Object storage
0G	TBD (publicly evolving)	TBD	TBD	TBD	AI-focused (per marketing)
Native (this paper)	Shared with PoUA, 2/3 honest	2D RS, k-sample	PoUA reputation-weighted	Block time (12s)	Attestation-shaped

Native DA matches Celestia’s security model (honest-majority + 2D RS sampling) while specializing the workload-fit dimension. EigenDA’s restaking-anchored security is Ethereum-coupled and not directly comparable to a Ligate-internal model. Avail’s KZG sub-second latency is a real improvement for some workloads; native DA does not match it but is workload-tuned in ways KZG isn’t. Walrus and 0G are not strong competitors at the security-and-attestation-fit intersection.

10.5 New Risks Specific to Specialization

A specialized from-scratch DA layer introduces three categories of risk that the Celestia-default does not:

(1) Implementation maturity. Celestia has years of operator experience and multiple external audits. Native DA inherits Sovereign SDK’s audit history but adds new attack surface in the per-schema commitment tree, signature aggregation, and tier-migration paths. Audit cost is real and ongoing; the first 1-2 years post-launch are at elevated risk.

(2) Validator-set bootstrapping. A new DA validator set needs geographic distribution, operator resilience, and slashing-budget calibration. Celestia’s validator set is already in this steady state; we’d be growing ours. Risk: under-decentralization in the first year (~50 validators instead of target 150) and lower honest-majority cushion.

(3) Sampling protocol bugs. 2D RS sampling is well-understood theoretically, but production implementations have had subtle bugs (e.g., Celestia’s namespace-sampling issue circa 2024). A from-scratch implementation will likely repeat some of those bugs. Mitigation: shared code with Celestia where possible, formal verification of sampling-critical paths, conservative parameter choices in the first year.

11. Comparison: Native vs Celestia vs Hybrid

11.1 Native vs Celestia

Side-by-side on the axes that matter:

- **Per-attestation cost.** Native: $\sim 280 \text{ B} \times \text{per-byte rate} + \text{protocol fee}$. Celestia: $\sim 4 \text{ KB} \times \text{Celestia per-byte rate (padded)}$ or $\sim 280 \text{ B} \times \text{Celestia per-byte rate (packed, but loses indexing)} + \text{protocol fee}$. Estimated 5-10x cost reduction on the DA-side for the padded baseline; 0-30% reduction vs the packed baseline depending on indexing-cost.
- **Latency to availability proof.** Both 12 seconds (matching block time). No improvement.
- **Query-shape match.** Native: $O(1)$ per attestor-history query. Celestia: requires off-DA index. Significant UX improvement for light clients.
- **Sovereignty.** Native: full fee-curve and roadmap control. Celestia: subject to Celestia governance.

11.2 Hybrid Mode

A hybrid deployment runs native DA for the hot tier (recent attestations, last 30 days) and Celestia for warm/cold archival. Reduces engineering surface (only the hot path is custom), keeps the bulk of historical data on a well-established DA layer.

Trade-offs. Reduces sovereignty (Celestia still controls warm/cold pricing). Adds operational complexity (two DA layers, tier migration path). Reduces audit burden (only hot tier is novel).

When hybrid makes sense. If §12 criteria partially fire (e.g., Celestia per-byte fees rise but governance stays Ligate-friendly), hybrid is the lower-risk path. If criteria fully fire (Celestia governance breaks alignment), hybrid is insufficient.

11.3 Engineering Cost

Honest estimate. Multi-quarter project, broken down approximately:

- **Q1:** Detailed protocol specification (this paper plus more), test-vector generation, simulator (prototypes/native-da-sim/).
- **Q2-Q3:** Rust implementation: validator-set bootstrap, per-schema commitment trees, BLS signature aggregation, 2D RS sampling, light-client protocol.
- **Q4:** Internal audit, testnet launch, validator-set growth.
- **Q5-Q6:** External audit (multi-vendor), mainnet launch, migration plan.

Roughly **4-6 person-quarters** of focused engineering, plus ~2 quarters of audit + operational ramp-up. At Ligate's team size (assuming team grew to 4-6 engineers by then), this is a 1.5-year horizon for full migration. The decision to start needs to be made 9+ months before migration is needed.

12. Migration Decision Criteria

12.1 When Migration Is Justified

Migration is justified if **at least two** of the following are simultaneously true at the time of evaluation:

1. **Celestia per-byte fees rise more than 4x from current levels** (or more than 2x during sustained-load operating periods). This signals Celestia's pricing has moved to a place where Ligate's attestation workload is being overcharged relative to the marginal cost.
2. **Celestia governance makes a roadmap decision incompatible with Ligate** (e.g., forks on a versioning question, changes the threshold, adopts a model that disadvantages attestation-shaped workloads). This signals strategic mis-alignment that's unlikely to be reversed.
3. **Workload measurement shows native DA would reduce attestation cost by more than 30%** at sustained-load operating point. This signals the engineering investment pays back in operational cost reduction.
4. **A binding technical incompatibility surfaces** (e.g., attestation throughput exceeds Celestia's near-term throughput target, light-client UX cost becomes a blocker for a flagship product launch).

Two-of-four is the bar deliberately: any single criterion could be transient or recoverable. Two simultaneous criteria indicate a deeper alignment shift.

12.2 When Migration Is NOT Justified

If none or only one of the §12.1 criteria are met, migration is not justified. The engineering cost (§11.3) exceeds the benefit at current trajectories. Default: **stay on Celestia**.

This is the expected state through v1 and likely v2. The paper exists so the decision can be made deliberately when criteria fire, not as a roadmap commitment.

12.3 Evaluation Cadence

Annual review. Every 12 months, the team evaluates against the §12.1 criteria and publishes the decision. The default is “stay”; the override is “evaluate migration.”

Out-of-cycle review. Any single §12.1 criterion firing (e.g., Celestia per-byte fees rise sharply) triggers an out-of-cycle review within 30 days. The review may decide to begin migration even with only one criterion firing if the criterion is severe enough (a 10x fee rise, for example).

13. Limitations and Future Work

13.1 Engineering Cost

This is the dominant limitation. Multi-quarter design + implementation + audit + bootstrap. Even when the technical case is strong, the engineering case must clear the §12 criteria. v0.2 documents the design; future work would be to refine the cost estimates as the chain matures.

13.2 Cross-DA Bridging

During migration, attestations would split across Celestia (legacy) and native DA (new). A bridging layer would let downstream consumers query across both DA sources. v0.2 sketches the bridging mechanism in §11.2 (hybrid mode) but defers full specification to a follow-up paper if migration is decided.

13.3 Quantum-Resistant Commitments

KZG depends on pairing-friendly elliptic curves; not post-quantum-resistant under Shor. Verkle depends on inner-product-argument schemes; also vulnerable to quantum attacks. The §6.1 default of namespace-aware Merkle trees (SHA-256 only) is post-quantum-resistant under Grover-bounded assumptions, but post-quantum signature schemes for the BLS aggregation in §6.3 are an active research area.

When (or if) the chain migrates to PQ-resistant cryptography (per native-delegation v0.2 §10.4 + per-schema-fees §9.5), the DA layer’s commitment scheme and signature aggregation must migrate in lockstep. Not v1 priority; named here for the migration roadmap.

13.4 Cross-Chain Attestation Portability

A native DA layer that does not bridge cleanly to other ecosystems (Ethereum, Cosmos via IBC, etc.) loses some composability benefits. A schema’s attestations on native DA could be made queryable from a counterparty chain via an IBC-mediated proof, but this adds protocol surface to the DA layer.

Cross-chain attestation portability is a research question larger than DA-layer choice; sketched in the native-delegation v0.2 §10.2 (cross-chain delegation), the per-schema-fees v0.2 §9.4 (cross-chain fee-market portability), and the cross-schema-composition v0.2 §9.1 (cross-chain composition). A unified cross-chain paper is the natural follow-up.

14. Conclusion

An attestation-optimized DA layer is technically tractable and would specialize in three dimensions Celestia does not: small-share efficiency (no 4 KB padding waste at attestation sizes), attester-history queries ($O(1)$ light-client proof vs $O(\text{epoch-shares})$ scanning), fee-market sovereignty (Ligate-controlled fee curve coupled to PoUA's τ_{burn} rebase).

The paper's four contributions resolve the design space. (1) **Workload characterization (§3)**: explicit modeling of attestation size distribution, throughput profile, ordering requirements, retention requirements, query patterns. (2) **Protocol specification (§5-§9)**: validator set, per-schema commitment trees with BLS signature aggregation, CometBFT-style consensus with 2D RS sampling, light-client protocol with attester-history queries, fee market with τ_{burn} coupling. (3) **Security analysis (§10)**: DA security under honest-majority + standard sampling assumptions; new-risk enumeration for the from-scratch design. (4) **Migration-decision criteria (§12)**: explicit thresholds (Celestia fee rises, governance alignment, cost-reduction estimates, technical incompatibilities) that would justify the engineering investment.

The mechanism is positioned as a **long-horizon migration target**, not a near-term replacement. Ligate Chain v0 and v1 run on Celestia. This paper exists so the future migration decision can be made deliberately, not reactively, and so the engineering work has a target if and when the decision fires.

What this paper does not do. It does not advocate for migrating off Celestia. It does not commit Ligate to building a native DA layer. It does not claim Celestia is failing. The §1.9 non-advocacy stance is deliberate; the §12 criteria are deliberately strict.

What this paper does do. Captures the design space at the point in time when the trade-offs are documentable, the comparison with prior systems is current, and the integration with companion primitives (PoUA, per-schema fees, native delegation, cross-schema composition, time-locked attestations) is concrete. If and when migration criteria fire, the engineering team has a reference document.

Invitations. Paper, future simulator, and (hypothetical) chain implementation are open to external review. Feedback on the §3 workload-model projections is especially welcome from operators of attestation-heavy workloads. Feedback on the §6 data-structure choices is welcome from DA-layer implementers (Celestia, EigenDA, Avail teams). Feedback on the §12 migration-decision criteria from operators who have run similar migration decisions on production chains is welcome.

The §1.4 central question was: what does an attestation-optimized DA layer look like, what does it cost to build relative to staying on Celestia, and under what conditions does the cost justify itself? This paper answers: per-schema commitment trees with BLS aggregation and PoUA-coupled fees, ~5 person-quarters of focused engineering, justified when 2 of 4 explicit criteria fire. Until they do, Ligate stays on Celestia.

References

Data availability layers.

- Al-Bassam, M., Sonnino, A., Buterin, V. (2018). *Fraud and Data Availability Proofs: Maximising Light Client Security and Scaling Blockchains with Dishonest Majorities*. <https://arxiv.org/abs/1809.09044>
- Al-Bassam, M. (2019). *LazyLedger: A Distributed Data Availability Ledger With Client-Side Smart Contracts*. <https://arxiv.org/abs/1905.09274> (Celestia precursor)
- Celestia documentation. <https://docs.celestia.org/>
- EigenLayer Labs (2024). *EigenDA: A New DA Layer for Ethereum Rollups*. <https://docs.eigenlayer.xyz/eigenda/overview>
- Avail Project (2024). *Avail Data Availability Network*. <https://docs.availproject.org/>
- Mysten Labs (2024). *Walrus: A Decentralized Storage and Data Availability Protocol*. <https://docs.walrus.site/>
- 0G Labs (2024). *0G: Programmable Data Availability for AI*. <https://docs.0g.ai/>

Erasure coding and sampling.

- Reed, I., Solomon, G. (1960). *Polynomial codes over certain finite fields*. SIAM Journal.
- McKenzie, W., White, B. (2024). *Practical 2D Reed-Solomon for blockchain data availability*. (Celestia engineering documentation.)

Commitment schemes.

- Kate, A., Zaverucha, G., Goldberg, I. (2010). *Constant-size commitments to polynomials and their applications*. ASIACRYPT 2010 (KZG).
- Kuszmaul, J. (2019). *Verkle trees*. <https://math.mit.edu/research/highschool/primes/materials/2018/Kuszmaul.pdf>
- Boneh, D., Lynn, B., Shacham, H. (2001). *Short signatures from the Weil pairing*. ASIACRYPT 2001 (BLS).

Permanent storage.

- Protocol Labs (2017). *Filecoin: A Decentralized Storage Network*. <https://filecoin.io/filecoin.pdf>
- Arweave (2018). *Arweave: A Protocol for Economically Sustainable Information Permanence*. <https://www.arweave.org/yellow-paper.pdf>

Companion Ligate Labs research.

- Ligate Labs (2026). *Proof of Useful Attestation*. Working paper v0.8.
- Ligate Labs (2026). *Native Delegation as a Runtime Primitive*. Working paper v0.2.
- Ligate Labs (2026). *Per-Schema Fee Markets*. Working paper v0.2.
- Ligate Labs (2026). *Cross-Schema Composition*. Working paper v0.2.
- Ligate Labs (2026). *Time-Locked Attestations*. Working paper v0.2.
- Ligate Labs (2026). *Schema-Bound Tokens*. Working paper v0.1.

Chain stack.

- Sovereign Labs (2024). *Sovereign SDK*. <https://github.com/Sovereign-Labs/sovereign-sdk>
- Celestia Labs (2023). *Celestia: Modular Data Availability*. <https://celestia.org/learn/>

- Inter-Blockchain Communication (IBC) protocol specification. <https://github.com/cosmos/ibc>

Appendix A: Simulator Validation Plan

A reference simulator under `prototypes/native-da-sim/` is planned for when (or if) §12 migration criteria fire. The simulator follows the v0.7-PoUA + v0.2 native-delegation + v0.2 per-schema-fees discipline: every numerical claim and every protocol-specification choice in this paper gets a corresponding simulator test or test vector.

Planned modules under `src/native_da_sim/`:

- `workload.py`: synthetic-attestation traffic generator matching §3’s size + throughput model. Generates devnet-realistic streams of attestations for testing the sampling and commitment paths.
- `validators.py`: §5.1 DA validator set, with configurable validator count, threshold, and sampling-bandwidth budget.
- `shares.py`: §5.2 2D Reed-Solomon erasure coding with configurable share size.
- `commitment_tree.py`: §6.1 per-schema commitment tree (namespace-aware Merkle, with optional KZG variant for benchmarking).
- `signature_aggregation.py`: §6.3 BLS aggregation per schema per epoch.
- `light_client.py`: §8 light-client query API (inclusion proofs, attester-history queries, epoch summaries).
- `fee_market.py`: §9 hybrid pricing model, integrated with the per-schema-fees-sim’s τ_{burn} coupling.

Planned test coverage:

- §3 workload-model regression tests (size distributions match the §3.1 table)
- §5.4 adversary scenarios (data-withholding, eclipse, sample-amplification)
- §6 data-structure correctness (Merkle proofs verify, BLS aggregates verify, tier-migration preserves commitments)
- §7 sampling-guarantee benchmarks (false-acceptance probability vs sample rate)
- §8 light-client proof-size + verification-cost benchmarks
- §9 fee-market interaction with the PoUA τ_{burn} rebase

Cross-language test vectors in the simulator’s `test_vectors/` directory, matching the format used by per-schema-fees-sim and native-delegation-sim.

The simulator is **not** part of v0.2 of this paper. It lands when §12 migration criteria fire; until then, the simulator-spec is the M1 milestone for the future engineering project.

Appendix B: Comparison Methodology

The §10.4 comparison table was constructed from publicly-available specifications of each system, normalized across different threat-model framings. Methodology notes:

Per-system sources. Celestia: official documentation as of mid-2026 + Al-Bassam et al. papers. EigenDA: EigenLayer Labs whitepaper + ongoing protocol updates. Avail: Polygon documentation

+ KZG benchmarks. Walrus: Mysten Labs documentation. 0G: published technical documents as of mid-2026 (caveat: 0G’s spec was still evolving as of the paper’s writing).

Normalization choices. “2/3 honest” is shorthand for various threshold conventions ($2f+1$, $f < n/3$, etc.) that resolve to the same security threshold; we use the most-common framing per system. “Sampling guarantee” is the system’s stated DA-correctness probability at its recommended sample-rate; different systems target different probabilities (2^{-32} false-acceptance is standard but not universal). “Security model” is the highest-level abstraction the system claims to inherit (honest-majority validators, restaking-bounded, validity-proof model, etc.).

Caveats for evolving systems. 0G’s specification was incomplete at the time of writing; the comparison-table entry reflects what was publicly documented as of mid-2026. EigenDA’s parameter choices have changed during its production lifetime; the table reflects the v1 generation. Future revisions of this paper will update.